

Agreement Observability for Byzantine Consensus: Estimation Sketches, Binding Accumulators, and a Safe Fast Path

Ryan Mercier

Abstract—Byzantine fault tolerant (BFT) protocols compare validator state with collision-resistant hashes, which reveal whether two views differ but not by how much. We study what a protocol gains from measuring *how much*, and what it must never do with that measurement. The measurement is a Rademacher sketch: each transaction maps to a pseudorandom vector in $\{-1, +1\}^n$ and a set to the integer sum, so the squared distance between two sketches over n is an unbiased estimate of the symmetric difference (the AMS tug-of-war estimator). We show that this sketch cannot be binding at any modulus, because its sums never wrap, and pair it with an LtHash accumulator that is; one ledger digest carries both. We then give an explicit polynomial-time attack that makes an arbitrarily large difference read as roughly a dozen transactions at $n = 512$, which settles where estimation belongs: scheduling, synchronization, and monitoring, never safety. On a two-phase locking BFT core with view change we add a speculative one-round fast path at the quorum $\lceil (n + 3f + 1)/2 \rceil$, prove and fuzz its safety (0 violations in 50,000 adversarial executions), and show that the $2f + 1$ fast path of our earlier version is unsafe across view changes (about 4% of fuzzed executions decide conflicting values). Around the core we build readiness sensing, partition detection and healing, $O(\log H)$ fork localization, and a cross-shard conservation invariant that detects dropped, replayed, minted, and misrouted credits exactly and localizes them by group testing. An honest evaluation against HotStuff, HotStuff-2, chained HotStuff, and Kauri finds no message or bandwidth advantage: the augmentation costs about 46% more bytes than HotStuff-2 at $N = 1000$ and concentrates load on the leader. What it buys is visibility into who is behind, by how much, and where, at a price we report exactly.

Index Terms—Byzantine fault tolerance, consensus, sketching, homomorphic hashing, set reconciliation, sharding, fast path.

I. INTRODUCTION

WHEN two validators hash their transaction sets and the hashes differ, a BFT protocol learns one bit. It does not learn whether one transaction is missing or half the block, whether one validator is behind or the network has split in two, or which of sixty cross-shard corridors lost a credit. Modern protocols route around this blindness well. HotStuff [2] and HotStuff-2 [3] vote on block hashes after dissemination; DAG protocols such as Narwhal [4], Bullshark [5], and Autobahn [6] separate data availability from ordering so that votes never wait on mempool convergence. None of them, however, can *observe* disagreement short of reconciling it, and several decisions

would benefit from observation: when to propose so that a one-round fast path will fire, whom to synchronize first, whether a stall is a slow validator or a partition, and which shard pair broke settlement.

This article asks what a protocol gains from a cheap, mergeable measurement of disagreement, and what it must never do with one. The answers are deliberately conservative, and several of them correct our earlier work [1].

The measurement and its limits (Sections IV, V). The measurement is the AMS tug-of-war sketch [7] with random-oracle signs: a set's sketch is the integer sum of per-item ± 1 vectors, and squared sketch distance over n is an unbiased estimate of the symmetric difference. The estimator is classical; PBS [8] already uses it to size set reconciliation. Our first observation concerns binding: because every coordinate of a ± 1 sum is bounded by the item count, reducing modulo q never wraps for any realistic set, a modular collision *is* an integer collision, and the modulus contributes nothing. The lattice analyses that protect LtHash [9] therefore do not transfer. We keep the sketch for estimation only and pair it with a uniform-coordinate LtHash accumulator for equality; a ledger digest carries both. Our second observation concerns robustness: an adversary who knows the salt and chooses which side of a difference each new item lands on can make any number of differing items read as a constant. At $n = 512$, greedy balancing over 256 candidates per step holds the estimate near 14 whether the true difference is 200 or 1600 (Fig. 1). This is an explicit, measured instance of known impossibility results for sketches under adversarial inputs [10]–[12], and it fixes the architecture: estimates may gate scheduling, synchronization, and monitoring, and must fail soft everywhere.

A safe fast path (Section VI). Estimation earns its keep where it decides *when* to try something optimistic. Our earlier version finalized in one round once $2N/3$ validators signed a proposal. That rule is safe within a view and unsafe across views: a fast decision seen by one honest validator can be invisible to the next leader. We place the fast path on a two-phase locking core with view change, use the fast quorum $n_{\text{fast}} = \lceil (n + 3f + 1)/2 \rceil$ of FaB [13] and SBFT [14], and give the view-change rule that recovers fast decisions. Our fuzzer, which plays equivocating leaders, double-voting replicas, and adversarial scheduling, finds conflicting decisions in about 4% of executions of the $2f + 1$ rule and none in 50,000 executions of the corrected protocol. It also found a bug in our own first draft of the corrected rule (Section VI). At optimal resilience $n = 3f + 1$ the safe fast quorum is all n validators; the fast

R. Mercier is with the School of Engineering, University of Connecticut, Storrs, CT, USA (e-mail: ryan.mercier@uconn.edu).

Code, tests, and every number in this article: <https://github.com/RyanMercier/Proxima>. An earlier version appeared at the 8th International Congress on Blockchain and Applications [1]. Section X lists the claims of that version this article corrects, including an unsafe fast path.

path is useful when the configured f is below $n/3$.

Observability (Sections VII, VIII). On that core the sketch supports readiness sensing (propose when the gossiped sketches say n_{fast} validators already hold the block), partition detection from bimodal sketch clouds with healing at cost proportional to the divergence, $O(\log H)$ fork localization on prefix accumulators, and a cross-shard conservation invariant. The last is double-entry bookkeeping with homomorphic digests: outbound and inbound corridor accumulators must balance once a settlement window has passed. Keying items by corridor and indexing them by debit height makes detection exact for dropped, replayed, minted, and misrouted credits and immune to honest settlement lag, and group testing localizes faulty corridors.

An honest evaluation (Section IX). Our earlier version reported message counts $2\times$ below HotStuff. Under one shared cost model, with certificates delivered to every validator and stragglers fetching missing transactions by id, the augmented protocol sends as many messages as HotStuff-2, about 46% more bytes at $N = 1000$, and loads its leader more; chained HotStuff is the cheapest protocol we measured. Conservation verification is cheaper than per-corridor batch roots only on dense corridor graphs (from 34 shards on a full mesh). We report these results because they locate the contribution correctly: observability, not efficiency.

All code, the 59 executable checks, and the script that produces every figure and number are public, and results are bit-reproducible.

II. RELATED WORK

BFT and fast paths. PBFT [15] established practical BFT; HotStuff [2] made it linear with leader-based aggregation, HotStuff-2 [3] removed a phase, and Tendermint [16] has a similar locking structure. One-round decisions descend from FaB Paxos [13], Zyzzyva [17], and SBFT [14]; Kuznetsov, Tonkikh, and Zhang [18] settle the resilience needed for two-step Byzantine consensus. Our fast-path quorum and view-change rule follow this line; our contribution is the estimation layer that decides when to attempt it, and a correction of our own earlier rule.

Dissemination and DAG protocols. Narwhal [4], Bullshark [5], and Autobahn [6] decouple data availability from consensus, so validators vote on certified batches rather than on synchronized mempools. This removes the throughput cost that motivated our earlier version and is the main reason we no longer claim efficiency. Observability composes with these designs: the same sketches can summarize which certified batches a validator holds. Kauri [19] aggregates over trees with reconfiguration on relay failure; our tree mode uses the same fallback.

Multiset hashing. Additive multiset hashes start with AdHash [20], whose moduli Wagner’s generalized birthday attack [21] forces to be large; Clarke et al. [22] formalize incremental multiset hashing, and LtHash [9] vectorizes it over $\mathbb{Z}_{2^{16}}^{1024}$ with an SIS-style analysis [23]. LtHash is deployed for update propagation at Meta and as Solana’s accounts hash [24], so homomorphic ledger digests inside consensus are not new.

Uniform coordinates make LtHash differences uniform, hence distance-free; this is why we carry a separate sketch.

Sketches under adversaries. The estimator is the AMS sketch [7] applied to set indicators. Mironov, Naor, and Segev [10] study sketching when the adversary knows the public randomness; Ajtai et al. [11] prove that white-box adversaries force linear space for F_2 approximation and show computational assumptions can restore robustness; Hardt and Woodruff [12] show linear sketches fail under adaptive queries. Robust property-preserving hashing [25]–[27] gives hashes whose distance predicates hold against adversaries, at output sizes growing with the threshold; Holmgren et al. pair a decodable syndrome with a homomorphic collision-resistant hash, the same pattern as our estimate-then-decode synchronization. Our deflation attack is a concrete, measured instance in the public-salt setting; we claim no new impossibility.

Set reconciliation. Characteristic polynomials [28], IBLTs [29], the Strata estimator [30], Graphene [31], Er-lay [32], and PBS [8] cover exact reconciliation and difference estimation; rateless IBLTs [33], CertainSync [34], and self-measuring IBLTs [35] remove the need for a separate estimate. Range-based reconciliation [36] bisects monoid fingerprints, the structure of our fork localization. Dold and Grothoff [37] run estimate-then-reconcile inside Byzantine set-union consensus.

Robust location and gossip. Coordinate-wise medians come from Byzantine-robust learning [38], [39] and multidimensional approximate agreement [40]; push-sum gossip [41] computes the averages our sketches admit.

Sharding. Receipt-based settlement [42] carries metadata proportional to volume; data-availability sharding [43] addresses a different layer. We found no prior use of homomorphic accumulators as a cross-shard conservation check.

III. MODEL

There are n validators, at most f Byzantine, with $n \geq 3f + 1$; the adversary is static and coordinates the Byzantine validators. Communication is partially synchronous [44]. Validators hold BLS [45] keys over BLS12-381; signatures on the same message aggregate. SHA-512 and SHAKE-256 are random oracles. A proposal lists the ids of its transactions; an honest validator that lacks some of them when the proposal arrives (probability p_{miss} , a swept parameter, missing at most $k_{\text{max}} = 2$) fetches them by id from the leader before voting. A block is a set and validation rejects duplicates.

IV. THE LEDGER DIGEST

A. Estimation: the Rademacher sketch

With salt s , each item x maps to $v_s(x) = \text{sgn}(H(\text{dom} \parallel s \parallel x)) \in \{-1, +1\}^n$, the first n bits of SHA-512 mapped to ± 1 , and a set T to $D_s(T) = \sum_{x \in T} v_s(x)$, computed over the integers and sent as n 16-bit words ($n = 512$, 1 KB). Sketches add, subtract, and aggregate exactly; no floating point enters any decision.

Lemma 1 (AMS estimator for sets): For sets A, B fixed before s is revealed, with $d = |A \triangle B|$ and $\Delta D = D_s(A) - D_s(B)$, the estimate $\hat{d} = \|\Delta D\|^2/n$ satisfies $\mathbb{E}[\hat{d}] = d$ and $\text{Var}(\hat{d}) = 2d(d-1)/n$, so its relative standard error is at most $\sqrt{2/n}$.

For multisets with multiplicity difference c , $\mathbb{E}[\hat{d}] = \|c\|_2^2$, not $\|c\|_1$.

Proof: Items in $A \cap B$ cancel; the d remaining vectors enter with fixed signs, and the coordinates of ΔD are independent sums of d Rademacher variables S with $\mathbb{E}S^2 = d$ and $\mathbb{E}S^4 = 3d^2 - 2d$. Summing over n coordinates gives the mean nd and variance $n(2d^2 - 2d)$ of $\|\Delta D\|^2$. With multiplicities the per-coordinate sum is $\sum_i c_i \varepsilon_i$, whose second moment is $\|c\|_2^2$. ■

The small- d laws are exact and serve as test vectors: $\|\Delta D\|^2 = n$ at $d = 1$; $4\text{Bin}(n, 1/2)$ at $d = 2$; $n + 8\text{Bin}(n, 1/4)$ at $d = 3$. A substitution of j items reads as $2j$ (Fig. 2); the nonnegative coordinates of our earlier version let substitutions partially cancel.

Straggler threshold. The integer test $\|\Delta D\|^2 \leq 3n$ classifies *honest* divergence from a reference. Computing the distributions exactly (Fig. 3), an honest view with $d = 2$ exceeds $3n$ with probability 1.6×10^{-31} (Hoeffding gives $e^{-n/8}$; the deviation is $n/4$ binomial units), a view with $d = 4$ falls at or below it with probability 4.9×10^{-7} , and $d = 3$ is a coin flip (0.48 above). At $n = 128$ the corresponding values are 2.1×10^{-9} and 8.3×10^{-3} . The threshold says nothing about Byzantine validators, who can submit any vector, including the reference itself.

Salts. Round sketches are salted with the height and the previous block hash, which nobody knows before the previous block is final, so no table can be built in advance. Lemma 1 requires the sets to be fixed before the salt is known; where a party chooses its set after seeing the salt, the estimate is advisory (Section V).

B. Binding: why the sketch cannot do it

A ledger needs an accumulator whose equality is binding: prefix digests for fork localization, conservation checks, and prune accounting (a node proves it pruned exactly P by showing $D(\text{before}) - D(\text{after}) = D(P)$). Our earlier drafts used the sketch modulo $q = 2^{32}$ and appealed to the SIS analysis of LtHash. That appeal fails.

Proposition 1 (The modulus is inert): Let V be any matrix with entries in $\{-1, +1\}$ and z an integer vector with $\|z\|_1 < q/2$. Then $Vz \equiv 0 \pmod{q}$ if and only if $Vz = 0$ over $\mathbb{Z}$.

Proof: Each coordinate of Vz has absolute value at most $\|z\|_1 < q/2$, and the only multiple of q in $(-q/2, q/2)$ is 0. ■

Every collision between realistic sets has $\|z\|_1 < 2^{31}$, so binding for the sketch is the problem of finding a signed subset of random ± 1 vectors that sums to zero over the integers. Lattice reduction is not the cheapest attack here; a generalized-birthday attack over integer sums is, and a first-cut estimate puts it near 2^{110} work when long histories allow large solution weights, far below the $n \log_2 q$ budget the earlier text implied. We did not pursue refinements. Instead the ledger digest binds with LtHash: each item expands through SHAKE-256 to a uniform vector in $\mathbb{Z}_{2^{1024}}$, exactly the parameters analyzed by Lewi et al. [9], whose security argument applies to sets verbatim. A *ledger digest* is the pair (LtHash, sketch): both are linear, equality is decided by LtHash alone, and the sketch reports

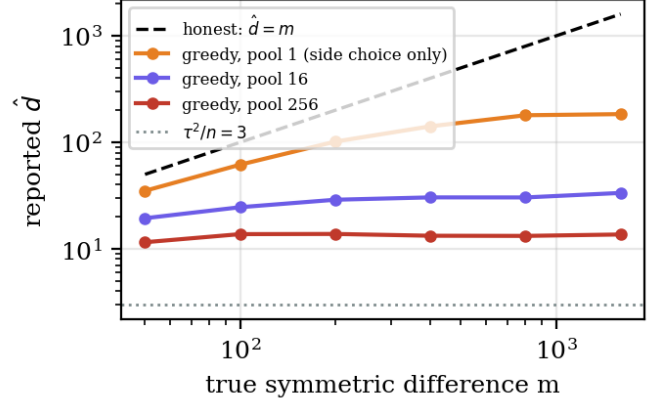


Fig. 1. Greedy deflation at $n = 512$: reported $\hat{d}$ against the true symmetric difference m for per-step search budgets $K = 1, 16, 256$. The estimate plateaus independent of m .

how large a difference is. It costs 4 KB on the wire (2 KB each).

V. WHAT THE SKETCH CANNOT PROMISE

Proposition 2 (Greedy deflation): An adversary who knows the salt can produce sets X, Y with $|X \Delta Y| = m$ for any m such that $\|D(X) - D(Y)\|^2/n$ stays below a level that depends on n and the adversary's per-step search budget K but not on m , using mK hash evaluations.

The construction keeps the running difference $S = D(X) - D(Y)$; at each step it draws K fresh items and adds the one, to whichever side, that most reduces $\|S\|$. A random vector's inner product with S has spread about $\|S\|$, so each step removes roughly the squared maximum of K such products and adds n ; the two balance at a steady-state norm independent of m . Fig. 1 measures it at $n = 512$: with $K = 256$ the estimate stays between 11 and 14 for true differences from 50 to 1600; even $K = 1$, where the adversary only chooses the side, plateaus near 180. The pair-grinding analysis in our earlier drafts, which bounded deflation to constant factors at exponential cost, considered the wrong attack.

Two consequences follow. First, no decision whose *safety* depends on the estimate is admissible; this is the trivial direction (sets at distance one already differ in a transaction a protocol may care about) made quantitative. Robust property-preserving hashes [25]–[27] show that a different object with a larger output can certify distance thresholds against adversaries; the AMS sketch is not that object. Second, every consumer of an estimate must fail soft. In our protocols an underestimate undersizes an IBLT, which then doubles (extra rounds); misleads the sensing proposer into proposing early (a lost fast path); or hides a partition until quorum failure reveals it (the status quo). Where an estimate must withstand chosen inputs, the parties should commit to their sets before the salt is drawn (*commit, then salt*), restoring the hypothesis of Lemma 1.

VI. CONSENSUS: A BASE PROTOCOL AND A SAFE FAST PATH

A. The base protocol II

II is single-slot agreement in the two-phase locking style of PBFT, Tendermint, and HotStuff-2. In view v the leader collects NEW-VIEW messages from $n - f$ validators, each carrying the sender's lock (the highest prepare certificate it holds) and its prepare-vote history; it proposes the value select forces, or any value if none is forced, attaching the NEW-VIEW set. Validators vote PREPARE once per view if the proposal equals select of the attached set and their lock permits it (the proposal matches the lock, or the forcing evidence is from a view at least as high). $n - f$ prepare votes form a prepare certificate, on which validators lock and vote COMMIT; $n - f$ commit votes decide.

B. The fast path

If one view gathers $n_{\text{fast}} = \lceil (n + 3f + 1)/2 \rceil$ prepare votes for a value, those votes are a decision certificate: one voting round. select must then recover fast decisions:

Definition 1 (select): Scan views from highest to lowest. At the first view holding a prepare certificate, return its value; at the first view in which some value has at least $n_{\text{fast}} - 2f$ claimed prepare votes in the NEW-VIEW set, return that value. If neither occurs, the leader is free.

Theorem 1 (Safety): If at most f validators are Byzantine, no two honest validators decide different values.

Proof sketch: Suppose x is decided first in view w . If by fast certificate, at least $n_{\text{fast}} - f$ honest validators voted x in w ; any $n - f$ NEW-VIEW set omits at most f of them, so x has at least $n_{\text{fast}} - 2f$ claims for view w , while any $y \neq x$ has at most f (Byzantine claims) plus $n - n_{\text{fast}}$ (honest votes cast elsewhere), which is below $n_{\text{fast}} - 2f$ because $2n_{\text{fast}} > n + 3f$. If by commit certificate, at least $n - 2f \geq f + 1$ honest validators lock on a prepare certificate for x from view w , and every $n - f$ set contains one. A prepare certificate and a fast certificate in the same view share at least $n_{\text{fast}} - f > f$ honest voters, so they name the same value. By induction over later views, honest validators vote only for x : any valid NEW-VIEW set forces x at view w , holds no certificate for another value in a later view, and holds at most $f < n_{\text{fast}} - 2f$ claims for one. Hence every later certificate, fast or slow, is for x . ■

Two details matter, and our fuzzer found both. The first is the quorum: with $n_{\text{fast}} = n - f = 2f + 1$, the rule of our earlier version, the counting step fails. A minimal counterexample: with $n = 4$, $f = 1$, a Byzantine leader equivocates, the Byzantine validator and two honest validators vote y , one honest validator sees the three votes and decides y ; in the next view the leader's NEW-VIEW set holds the Byzantine (now claiming x), the honest x -voter, and one y -voter, so x is forced and the slow path decides x . The second is the history: our first implementation of the corrected rule sent only each validator's latest vote. At $n = 6$, $f = 1$ the fuzzer found an execution in which honest validators kept voting x in later low-turnout views, each below threshold, overwriting the view- w evidence until a leader was free to propose y . Reporting the vote history (in a deployment, the votes cast since the validator's lock), as FaB and SBFT do, closes it. Table I summarizes the fuzzing:

TABLE I
ADVERSARIAL FUZZING OF THE BASE PROTOCOL WITH THE FAST PATH
(10,000 EXECUTIONS PER ROW, f BYZANTINE).

n	f	n_{fast}	rule	conflicting	not live
4	1	4	$\lceil (n + 3f + 1)/2 \rceil$	0	0
6	1	5	$\lceil (n + 3f + 1)/2 \rceil$	0	0
7	2	7	$\lceil (n + 3f + 1)/2 \rceil$	0	0
9	2	8	$\lceil (n + 3f + 1)/2 \rceil$	0	0
10	3	10	$\lceil (n + 3f + 1)/2 \rceil$	0	0
4	1	3	$n - f$ (earlier version)	363	0
7	2	5	$n - f$ (earlier version)	434	0
10	3	7	$n - f$ (earlier version)	425	0

10,000 executions per configuration with equivocating leaders, double-voting validators that lie in NEW-VIEW, random partial delivery for four asynchronous views, then synchronous views until an honest leader decides.

The price. At $n = 3f + 1$ the safe fast quorum is n : a single abstaining validator disables the fast path, as for every one-round protocol at optimal resilience [18]. The fast path pays when deployments configure f below $n/3$; with $n = 100$ and $f = 20$, $n_{\text{fast}} = 81$, so up to 19 validators may be slow or abstain.

Where the sketch enters. Prepare votes may carry the voter's sketch of the proposal; the reference is $D(B)$, which anyone computes from the block, so no party has discretion over it. The sketches report who was behind and by how much; they never gate a vote or a certificate. The fast-path trigger is a signature count, so a Byzantine validator that signs correctly but submits a misleading sketch cannot suppress it. The variance trigger of our earlier version could be suppressed by one such validator (Fig. 4, left).

Proposition 3 (Liveness after GST): After GST, a view with an honest leader decides through the slow path: the $n - f$ honest validators obtain missing transactions by id within the view timeout and vote. The fast path fires iff at least n_{fast} valid prepare votes arrive before the fast timeout; with b correctly signing Byzantine validators and each of h honest validators independently holding the full block with probability $1 - p_{\text{miss}}$, this is $\Pr[\text{Bin}(h, 1 - p_{\text{miss}}) \geq n_{\text{fast}} - b]$.

The liveness theorem of our earlier version bounded the failure probability by $\exp(-2N(1/3 - p)^2)$, which ignores the f Byzantine abstainers: at $f = \lfloor (N - 1)/3 \rfloor$ the honest validators *are* the quorum, and with $p = 0.01$ at $N = 1000$ the probability that at least one of them is missing at a fixed deadline exceeds 0.99, not 10^{-91} . Correct statements count the slack $n - f - q$ explicitly, as above, and rely on synchronization rather than on luck.

Tree mode. Validators form leaves of ten; the first member of each leaf is its leader, and validators are placed at random, so some leaf leaders are Byzantine. Votes aggregate up the tree; a leaf whose leader is silent times out and its members send directly to the root, as in Kauri. Leaf summaries of sketches are exact integers.

VII. OBSERVABILITY

Readiness sensing. Each gossip interval, validators send the proposer the sketch of their view of the candidate block

(1 KB each, charged in full). The proposer proposes when at least n_{fast} sketches equal $D(B)$ exactly, or at a cap. Byzantine validators can report $D(B)$ without holding the block; the effect is an early proposal and a lost fast path, never a safety issue. The comparison that matters is against a fixed delay of the same mean, not against proposing immediately.

Synchronization. Inside consensus a validator knows the ids it lacks and fetches them; no sketch is needed (Fig. 5, left). Sketch-sized IBLTs [29] matter where no id list exists: mempool reconciliation between peers, partition healing, and fork healing. IBLT short ids are salted per session, as in Erelay, so an adversary cannot grind colliding ids in advance; the decoder knows only its own items and fetches the rest in one further round.

Partition detection and healing. A partition makes the sketch cloud bimodal. Camp centroids are exact by linearity, the inter-camp $\hat{d} = \|\mu_1 - \mu_2\|^2/n$ sizes the repair before it starts, and one estimate-then-decode exchange between representatives gives both camps the union.

Fork localization. Prefix accumulators $P[b] = \sum_{b' \leq b} D(\text{block}_{b'})$ make every range digest a subtraction. Two nodes find their fork point by binary search on LtHash equality of prefixes in $\lceil \log_2 H \rceil + 1$ probes, and each probe's sketch reports how much diverged. The structure is that of range-based set reconciliation [36]; the size estimate per probe is the addition.

Gossip-averaged agreement. Because sketches are linear, push-sum gossip [41] lets every validator estimate the mean sketch μ and the dispersion $\frac{1}{N} \sum_j \|D_j - \mu\|^2/n$ without an aggregator. The dispersion plays the role of $1 - r^2$ for the Kuramoto order parameter r of coupled oscillators, which inspired this project's name; we claim the analogy as a metric only, and a Byzantine-robust gossip variant is future work.

VIII. CROSS-SHARD CONSERVATION

Each cross-shard transaction t from shard i to shard j , debited at block h , adds the ledger digest of the corridor-keyed item (i, j, t) to i 's outbound accumulator $O_{ij}[h]$; when j applies the credit, whose message carries h , the same item enters j 's inbound accumulator $I_{ij}[h]$. Each block, every shard's quorum signs its aggregate outbound and inbound digests over settled cohorts ($h \leq \text{now} - W$), and the beacon checks

$$\sum_i O_i^{\leq \text{now} - W} = \sum_j I_j^{\leq \text{now} - W}. \quad (1)$$

Threat model. Each shard's quorum attests honestly to what the shard applied; shard-internal execution is secured by its own BFT instance. The faults detected are in the cross-shard path: dropped, replayed, minted, and misrouted credits from faulty or Byzantine relayers and bridges. A Byzantine shard quorum could sign accumulators that misreport its own execution; catching that needs state-validity or fraud proofs and is out of scope.

Theorem 2 (Conservation): Under this threat model, if the settled invariant holds then the multiset of settled debits equals the multiset of settled credits, per corridor, unless an LtHash collision has been found.

Proof: Both sides are LtHash sums over corridor-keyed items. If the multisets differ, the two sums are a collision for distinct multisets of LtHash inputs. ■

Three design choices carry the result, and each repairs a failure of our earlier drafts. *Corridor keying:* without it, a credit applied on the wrong corridor adds the same vector the debit added and the global invariant still balances; with it, a misroute is a drop on one corridor and a mint on another. *Cohort indexing:* credits trail debits by network latency, so comparing whole accumulators raises alarms on every busy corridor; comparing only settled cohorts raises none. *Group testing:* on violation the beacon bisects the corridor set with aggregated settled digests; corridor keying makes cancellation inside a group require a collision, so the recursion is exact and uses at most $2f \lceil \log_2(C/f) \rceil + 1$ probes for f faulty corridors among C . A multiset-aware IBLT over the settled debits and credits of an alarmed corridor then names its dangling transactions; a replayed credit decodes as one extra.

The semantics are detection and settlement verification, not atomicity: pair the invariant with optimistic execution and a revert window, or credit only on match.

IX. EVALUATION

Method. The artifact accounts every message under one cost model shared by all protocols: the proposal (64 B plus 32 B per transaction id) reaches all N validators, votes are 128 B, a certificate is a 96 B aggregate plus 64 B of metadata and an $N/8$ -byte signer bitmap (or no bitmap with threshold signatures), stragglers fetch by id at 232 B per transaction, Byzantine validators abstain, every certificate reaches every validator, and we track bytes arriving at the leader. Signatures are verified: a hash-based verifiable mock at scale, real BLS12-381 in the tests. This is a counting simulator, not a network emulator; it reports no latency. Unless stated, 30% of validators are Byzantine at random positions, $p_{\text{miss}} = 0.37$, and blocks carry 21 transactions. Baselines are HotStuff, HotStuff-2, chained HotStuff amortized per block, a message model of Kauri (fanout 10, without pipelining or reconfiguration), and PBFT.

A. The estimator and the threshold

Fig. 2 shows the estimate for missing and substituted transactions against the analytic mean; at $d = 1$ the estimate is exactly 1 in all 400 trials. Fig. 3 plots the exact distributions behind the tail probabilities of Section IV.

B. Fast path

Fig. 4 (left) uses $N = 100$, $f = 20$, $n_{\text{fast}} = 81$, and complete views. Byzantine validators that sign correctly but submit misleading sketches disable the variance trigger of our earlier version at the first such validator and leave the signature trigger at 100% through 20 of them. Abstaining Byzantine validators leave it at 100% through 19 and disable it at 20, where fewer than n_{fast} signers remain; no trigger can do better. Fig. 4 (right) compares sensing with a fixed delay equal to the mean sensed delay (40 trials per point, gossip fill probability 0.5 per tick, cap 8). Sensing engaged the fast path in every

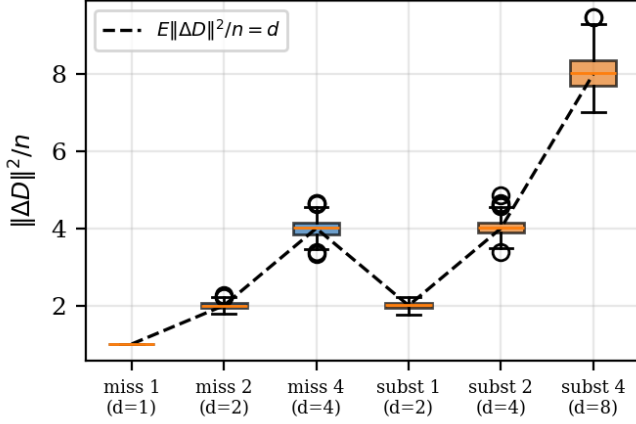


Fig. 2. $\|\Delta D\|^2/n$ for missing (blue) and substituted (orange) transactions, 400 trials per case, $n = 512$, against $\mathbb{E}\hat{d} = d$.

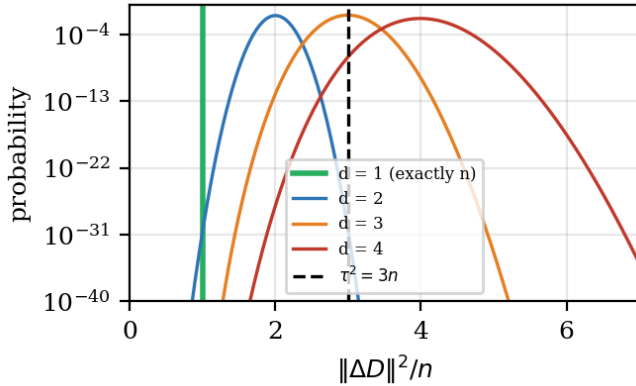


Fig. 3. Exact distributions of $\|\Delta D\|^2/n$ for $d = 1, \dots, 4$ at $n = 512$ (log scale) against $\tau^2 = 3n$.

trial at mean delays of 0 to 3.05 ticks; the fixed delay reached 55% to 100%, losing most where the mean delay rounds to zero ticks (55% at $p_{\text{miss}} = 0.2$). The gain comes from adapting the delay per block. Its cost is 100 KB of sketch gossip per tick at $N = 100$.

C. Synchronization

Fig. 5 (left) is the block-level case: fetching k missing transactions by id costs $232k$ bytes, an IBLT sized for k plus the push costs more, and a 1% Bloom filter over the 21-transaction block plus the push costs slightly less but admits false positives. No sketch is needed. The right panel is mempool reconciliation between two peers holding 5000 transactions. A sketch-sized IBLT decoded at its first size at every difference tested, so it costs the 1 KB sketch plus about what an IBLT sized by an oracle costs; starting from the minimum size and doubling is cheaper for $d \leq 8$ and costs $2.5\times$ more bytes and 8 rounds at $d = 512$. A Bloom filter of the whole set (5990 B) is cheaper than any IBLT beyond a few hundred differences, at the price of false positives. When a sketch is already on the wire it removes the doubling; self-sizing and rateless IBLTs [33]–[35] remove it otherwise.

TABLE II
PER-BLOCK COST AT $N = 1000$ (30% BYZANTINE ABSTAINING, $p_{\text{miss}} = 0.37$). KB WITH BITMAP AND THRESHOLD CERTIFICATES; LEADER INGRESS WITH BITMAPS.

protocol	msgs	KB (bitmap)	KB (thresh.)	leader KB
Proxima flat	4,661	2,239	1,995	875
Proxima flat, no sketches	4,661	1,539	n/r	175
Proxima tree	5,742	2,931	2,687	341
HotStuff	6,359	1,901	1,535	263
HotStuff-2	4,659	1,535	1,291	175
Chained HotStuff (amortized)	1,959	1,169	1,047	88
Kauri (message model)	7,252	2,041	1,635	7
PBFT	1,999,000	250,469	n/a	n/a

The no-sketch row is the same protocol with observability off; its leader ingress equals HotStuff-2's two vote phases. n/r: not run.

D. Conservation

In a 16-shard ring run with 5120 cross-shard transactions, credits landing one block after debits (640 still in flight at the check), and six injected faults (two drops, a replay, a mint, and a misroute), the settled invariant failed with a dangling estimate of 5.9 against 6; the aged alarms named exactly the five affected corridors (the misroute implicates two), bisection found them in 21 probes, and the decode recovered every dangling transaction exactly. The same run without faults raised no alarm. Fig. 6 (right) shows localization probes at $C = 64$ against the bound; at $f = 16$ bisection used 71 probes, more than testing all 64 corridors, which is the expected regime change for group testing. Fig. 6 (left) compares verification metadata. Conservation costs two signed 4 KB ledger digests per shard per block, independent of volume and of the corridor graph. Per-corridor batch roots cost 256 B per corridor; on a ring they are 32 times cheaper at every size; on a full mesh, conservation is cheaper from $S = 34$ shards on. Batched receipts grow with volume on top of the per-corridor roots.

E. Cost against the baselines

Table II and Fig. 7 report per-block cost. Messages: the augmented flat protocol matches HotStuff-2 (4661 against 4659 at $N = 1000$) because its slow path is HotStuff-2's two phases; it is below three-phase HotStuff and Kauri, and chained HotStuff is cheapest at 1959. Bytes: sketches on prepare votes cost about 700 KB per block at $N = 1000$ (2239 KB with sketches, 1539 KB without, bitmap certificates), putting the augmented protocol 46% above HotStuff-2. Threshold certificates remove the $N/8$ bitmaps from every protocol and leave the ordering unchanged. Leader ingress: every prepare vote carries 1 KB of sketch, so the flat leader receives 875 KB per block at $N = 1000$ against 175 KB for HotStuff-2; the tree mode moves that load to leaf leaders (341 KB at the root, including fallback traffic from Byzantine leaf leaders). The lighter $n = 128$ profile cuts sketch bytes by four at 12.5% estimator error.

F. Fork localization and healing

Fig. 8: fork localization used 7, 9, 11, and 13 probes at heights 64 to 4096, one 4 KB ledger digest each. Partition

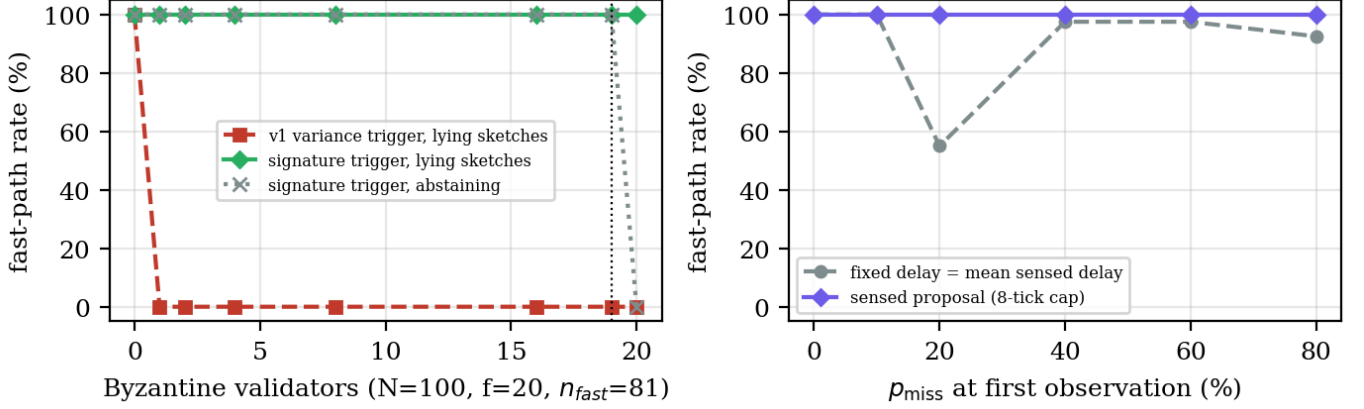


Fig. 4. Left: fast-path rate against the number of Byzantine validators ($N = 100$, $f = 20$, $n_{\text{fast}} = 81$, dotted line at $N - n_{\text{fast}}$): signing with misleading sketches versus abstaining. Right: fast-path rate against initial p_{miss} , sensed proposal versus a fixed delay of the same mean.

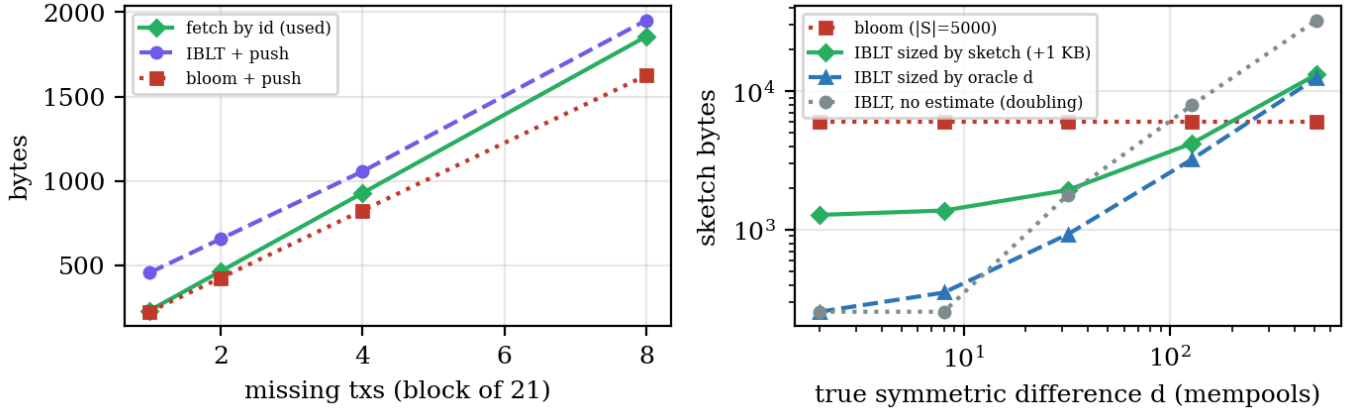


Fig. 5. Left: block-level synchronization of k missing transactions. Right: mempool reconciliation bytes (excluding the transfer of the missing bodies) against the true difference, 5000 transactions.

healing over 600 transactions recovered the union at every divergence from 2 to 128, with the pre-repair estimate within its predicted error (128.9 at $d = 128$) and total cost near 230 bytes per divergent transaction, including the second round in which the requester fetches the items it lacked.

X. CORRECTIONS TO THE EARLIER VERSION

For readers of [1] and of our earlier drafts:

- 1) The one-round fast path at $2N/3$ signatures is unsafe across view changes (Section VI, Table I). The safe quorum is $\lceil (n + 3f + 1)/2 \rceil$ with vote histories in NEW-VIEW.
- 2) The liveness bound $\exp(-2N(1/3 - p)^2)$ ignores Byzantine abstainers and is wrong near $f = N/3$ (Section VI).
- 3) Message counts $2\times$ below HotStuff did not deliver certificates to all validators, charged different retransmission costs, and omitted chained HotStuff; under a shared model there is no message advantage over HotStuff-2 (Section IX).
- 4) The aggregator-chosen and, in our drafts, median references give the aggregator discretion; inside consensus the reference is $D(B)$.

- 5) Tree leaves placed Byzantine validators last, so no leaf leader was ever Byzantine; with random placement leaves need a fallback path.
- 6) Binding of the ± 1 accumulator cannot rest on the modulus (Proposition 1); the ledger now binds with LtHash.
- 7) Deflation is not bounded by constant factors (Proposition 2).

XI. LIMITATIONS AND FUTURE WORK

No network measurements. All results are counts from a simulator; latency, throughput, and the real distribution of p_{miss} need a multi-region deployment.

Proofs. Theorem 1 is a proof sketch backed by fuzzing of single-slot agreement; a mechanized proof (TLA+ or Ivy) and a multi-slot, pipelined variant are open.

Robust estimation. Commit-then-salt restores unbiasedness against chosen sets; integrating it into gossip without an extra round, and Byzantine-robust push-sum over 512-dimensional sketches, are open.

Applications. The most promising use we see outside this paper is censorship monitoring for Ethereum inclusion lists [46]:

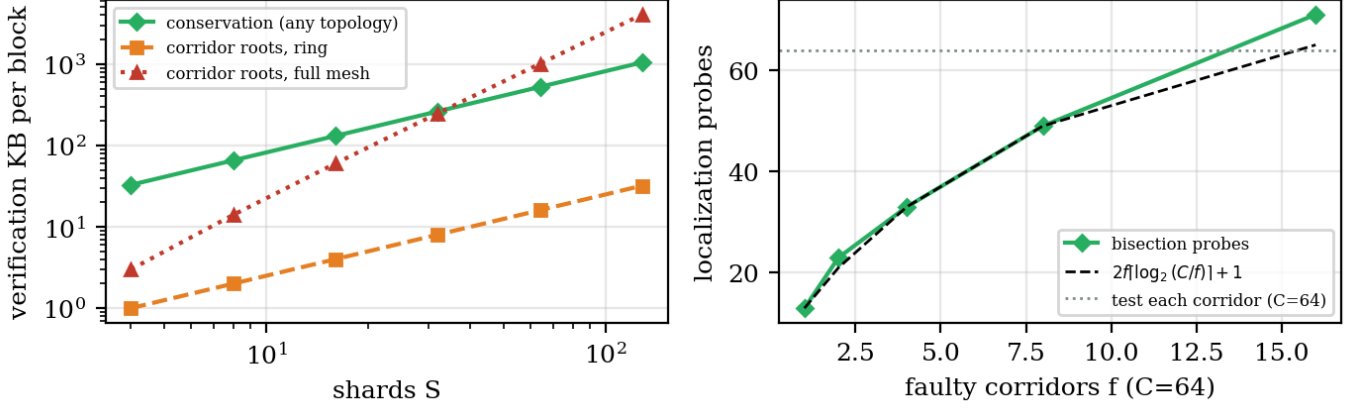


Fig. 6. Left: verification KB per block against shard count for conservation and for per-corridor batch roots on ring and full-mesh corridor graphs. Right: bisection probes against faulty corridors at $C = 64$, with the bound and the cost of individual testing.

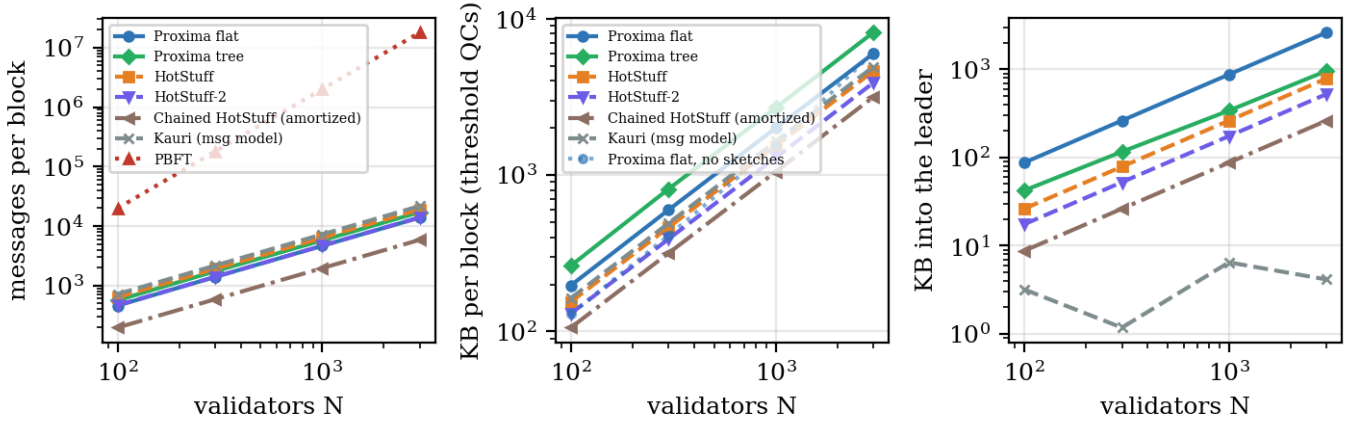


Fig. 7. Messages (left), bytes with threshold certificates (center), and bytes arriving at the leader (right) per block, log-log.

committee members publish sketches of the transactions they consider eligible, the median estimates how many a block omitted, and full lists are sent only when that estimate trips. Mempool divergence already causes false censorship accusations [47]; sketches measure it. Replica-consistency checks that report divergence size, and relay completeness scoring for gossip networks, are similar.

XII. CONCLUSION

A hash tells a consensus protocol whether views differ; a sketch tells it roughly how much, and an adversary who knows the salt can make that estimate say almost anything. Taking both facts seriously gives a narrow, useful role for measurement in BFT: deciding when to try a fast path, whom to synchronize, and where a partition or a broken shard corridor is, with every decision failing soft and every safety property resting on signatures and on an LtHash accumulator. Building that role carefully exposed an unsafe fast path, a wrong liveness bound, an inert modulus, and an evaluation that flattered our earlier work; the corrected system costs more bytes than HotStuff-2 and is worth it only where its observability is.

REFERENCES

- [1] R. Mercier, "Distance-preserving digests: A primitive for BFT consensus," in *Proc. 8th International Congress on Blockchain and Applications (BLOCKCHAIN)*. Springer LNNS, 2026, arXiv:2605.15329.
- [2] M. Yin, D. Malkhi, M. K. Reiter, G. Golan-Gueta, and I. Abraham, "HotStuff: BFT consensus with linearity and responsiveness," in *Proc. PODC*, 2019.
- [3] D. Malkhi and K. Nayak, "HotStuff-2: Optimal two-phase responsive BFT," in *Cryptology ePrint Archive, Paper 2023/397*, 2023.
- [4] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, "Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus," in *Proc. EuroSys*, 2022.
- [5] A. Spiegelman, N. Girdharan, A. Sonnino, and L. Kokoris-Kogias, "Bullshark: DAG BFT protocols made practical," in *Proc. CCS*, 2022.
- [6] N. Girdharan, F. Suri-Payer, I. Abraham, L. Alvisi, and N. Crooks, "Autobahn: Seamless high speed BFT," in *Proc. ACM SOSP*, 2024.
- [7] N. Alon, Y. Matias, and M. Szegedy, "The space complexity of approximating the frequency moments," in *Proc. STOC*, 1996.
- [8] L. Gong, Z. Liu, L. Liu, J. Xu, M. Ogiwara, and T. Yang, "Space- and computationally-efficient set reconciliation via parity bitmap sketch (PBS)," *Proc. VLDB Endowment*, vol. 14, no. 4, pp. 458–470, 2020.
- [9] K. Lewi, W. Kim, I. Maykov, and S. Weis, "Securing update propagation with homomorphic hashing," *Cryptology ePrint Archive, Paper 2019/227*, 2019.
- [10] I. Mironov, M. Naor, and G. Segev, "Sketching in adversarial environments," *SIAM J. Comput.*, vol. 40, no. 6, 2011.
- [11] M. Ajtai, V. Braverman, T. S. Jayram, S. Silwal, A. Sun, D. P. Woodruff, and S. Zhou, "The white-box adversarial data stream model," in *Proc. ACM PODS*, 2022.

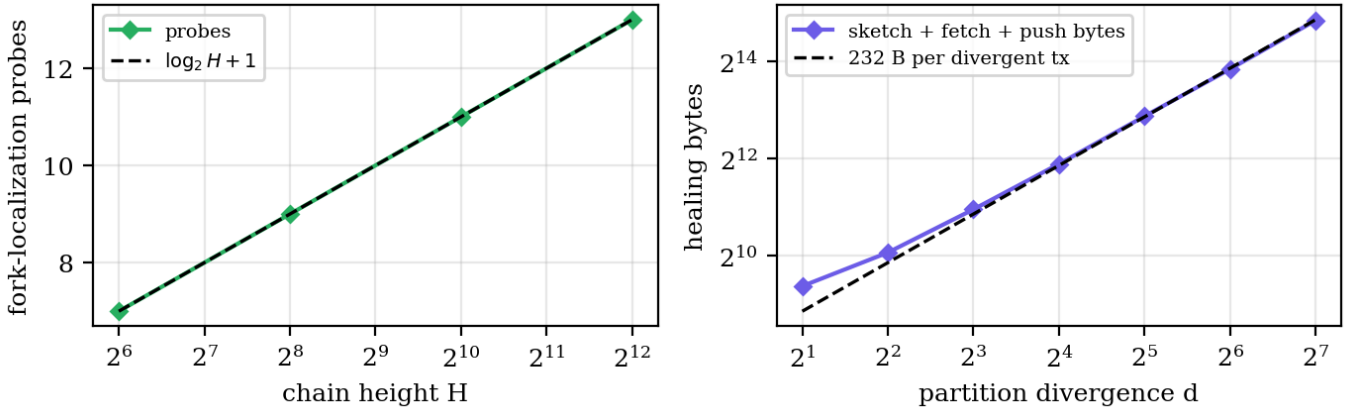


Fig. 8. Left: fork-localization probes against chain height. Right: partition-healing bytes against divergence.

- [12] M. Hardt and D. P. Woodruff, “How robust are linear sketches to adaptive inputs?” in *Proc. STOC*, 2013.
- [13] J.-P. Martin and L. Alvisi, “Fast Byzantine consensus,” *IEEE Trans. Dependable Secure Comput.*, vol. 3, no. 3, 2006.
- [14] G. Golan-Gueta, I. Abraham, S. Grossman, D. Malkhi, B. Pinkas, M. K. Reiter, D.-A. Seredinschi, O. Tamir, and A. Tomescu, “SBFT: A scalable and decentralized trust infrastructure,” in *Proc. DSN*, 2019.
- [15] M. Castro and B. Liskov, “Practical Byzantine fault tolerance,” in *Proc. OSDI*, 1999.
- [16] E. Buchman, “Tendermint: Byzantine fault tolerance in the age of blockchains,” Ph.D. dissertation, University of Guelph, 2016.
- [17] R. Kotla, L. Alvisi, M. Dahlin, A. Clement, and E. Wong, “Zyzyva: Speculative Byzantine fault tolerance,” in *Proc. SOSP*, 2007.
- [18] P. Kuznetsov, A. Tonkikh, and Y. X. Zhang, “Revisiting optimal resilience of fast Byzantine consensus,” in *Proc. ACM PODC*, 2021.
- [19] R. Neiheiser, M. Matos, and L. Rodrigues, “Kauri: Scalable BFT consensus with pipelined tree-based dissemination and aggregation,” in *Proc. SOSP*, 2021.
- [20] M. Bellare and D. Micciancio, “A new paradigm for collision-free hashing: Incrementality at reduced cost,” in *Proc. EUROCRYPT*, 1997.
- [21] D. Wagner, “A generalized birthday problem,” in *Proc. CRYPTO*, 2002, pp. 288–304.
- [22] D. Clarke, S. Devadas, M. van Dijk, B. Gassend, and G. E. Suh, “Incremental multiset hash functions and their application to memory integrity checking,” in *Proc. ASIACRYPT*, 2003.
- [23] M. Ajtai, “Generating hard instances of lattice problems,” in *Proc. STOC*, 1996.
- [24] Solana Foundation, “SIMD-0215: Accounts lattice hash,” <https://github.com/solana-foundation/solana-improvement-documents>, 2025.
- [25] E. Boyle, R. LaVigne, and V. Vaikuntanathan, “Adversarially robust property-preserving hash functions,” in *Proc. ITCS*, 2019.
- [26] N. Fleischhacker and M. Simkin, “Robust property-preserving hash functions for Hamming distance and more,” in *Advances in Cryptology – EUROCRYPT*, 2021.
- [27] J. Holmgren, M. Liu, L. Tyner, and D. Wichs, “Nearly optimal property preserving hashing,” in *Advances in Cryptology – CRYPTO*, 2022.
- [28] Y. Minsky, A. Trachtenberg, and R. Zippel, “Set reconciliation with nearly optimal communication complexity,” *IEEE Transactions on Information Theory*, vol. 49, no. 9, pp. 2213–2218, 2003.
- [29] M. T. Goodrich and M. Mitzenmacher, “Invertible bloom lookup tables,” *Proc. Allerton Conference on Communication, Control, and Computing*, 2011.
- [30] D. Eppstein, M. T. Goodrich, F. Uyeda, and G. Varghese, “What’s the difference? Efficient set reconciliation without prior context,” in *Proc. ACM SIGCOMM*, 2011.
- [31] A. P. Ozisik, G. Andresen, B. N. Levine, D. Tapp, G. Bissias, and S. Katkuri, “Graphene: Efficient interactive set reconciliation applied to blockchain propagation,” *Proc. ACM SIGCOMM*, 2019.
- [32] G. Naumenko, G. Maxwell, P. Wuille, A. Fedorova, and I. Beschastnikh, “Erlay: Efficient transaction relay for Bitcoin,” in *Proc. ACM CCS*, 2019.
- [33] L. Yang, Y. Gilad, and M. Alizadeh, “Practical rateless set reconciliation,” in *Proc. ACM SIGCOMM*, 2024.
- [34] T. Keniagin, E. Yaakobi, and O. Rottenstreich, “CertainSync: Rateless set reconciliation with certainty,” arXiv:2504.08314, 2025.
- [35] M. Wu, J. Qi, Z. Ye, C. Luo, S. Lu, and Z. Wei, “IBLTs measure before they decode: Self-sizing set reconciliation for database consistency verification,” arXiv:2608.26537, 2026.
- [36] A. Meyer, “Range-based set reconciliation,” in *Proc. IEEE SRDS*, 2023.
- [37] F. Dold and C. Grothoff, “Byzantine set-union consensus using efficient set reconciliation,” *EURASIP J. Information Security*, 2017.
- [38] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine learning with adversaries: Byzantine tolerant gradient descent,” in *Proc. NeurIPS*, 2017.
- [39] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in *Proc. ICML*, 2018.
- [40] H. Mendes and M. Herlihy, “Multidimensional approximate agreement in Byzantine asynchronous systems,” in *Proc. ACM STOC*, 2013.
- [41] D. Kempe, A. Dobra, and J. Gehrke, “Gossip-based computation of aggregate information,” in *Proc. IEEE FOCS*, 2003.
- [42] A. Skidanov, “Nightshade: Near protocol sharding design,” <https://pages.near.org/papers/nightshade/>.
- [43] Ethereum Foundation, “Danksharding,” <https://ethereum.org/roadmap/danksharding>.
- [44] C. Dwork, N. Lynch, and L. Stockmeyer, “Consensus in the presence of partial synchrony,” *Journal of the ACM*, vol. 35, no. 2, pp. 288–323, 1988.
- [45] D. Boneh, B. Lynn, and H. Shacham, “Short signatures from the Weil pairing,” *Journal of Cryptology*, vol. 17, no. 4, pp. 297–319, 2004.
- [46] Ethereum Foundation, “EIP-7805: Fork-choice enforced inclusion lists (FOCIL),” <https://eips.ethereum.org/EIPS/eip-7805>, 2024.
- [47] J. Albrecht and G. Karame, “On the effectiveness of mempool-based transaction auditing,” arXiv:2601.14996, 2026.